



UNIVERSIDAD
AUTÓNOMA
METROPOLITANA
Unidad Cuajimalpa

Diseño base de datos para Sistemas de Información

Guillermo Monroy Rodríguez

Doctor en Ciencias en Computación • gmonroy@cua.uam.mx

Evaluación del curso I

Aspectos a evaluar:

Cualitativos. Elementos que se deben cumplir para tener derecho a aprobar el curso:

- ▶ Asistencia a las sesiones | 100% de asistencia
- ▶ Participación activa en las sesiones | ≥ 1 participación activa como mínimo por sesión

Cuantitativos. Elementos que se deben cumplir para aprobar el curso. Se deberán aprobar todos los rubros para que se pueda aplicar el promedio.

- ▶ Tareas y ejercicios en sesión | el estudiante deberá hacer todas las tareas y ejercicios propuestos en cada sesión, para que se pueda obtener su diploma.
- ▶ Evaluación continua | será observado el grado de atención dirigida y el grado de aprendizaje

Condiciones para aprobar el curso

1. Asistencia a todas las sesiones
2. Participación acertiva en las sesiones
3. Realización de todas las tareas y ejercicios propuestos en cada sesión

Tema 1

Conceptos básicos

Conceptos básicos I

¿Qué es una base de datos?

- Una base de datos es un conjunto de datos que se almacenan de forma estructurada y organizada, permitiendo su fácil acceso, manipulación y actualización.
- Las bases de datos son fundamentales para el almacenamiento y recuperación eficiente de información en aplicaciones y sistemas informáticos.
- Pueden ser utilizadas en una amplia variedad de contextos, desde aplicaciones empresariales hasta sistemas web y móviles.
- Las bases de datos pueden ser clasificadas en diferentes tipos, como bases de datos relacionales, no relacionales, orientadas a objetos, entre otras.
- Las bases de datos son gestionadas por sistemas de gestión de bases de datos (SGBD), que proporcionan herramientas para crear, leer, actualizar y eliminar datos.

Conceptos básicos II

¿Qué es una base de datos?

- Las bases de datos son esenciales para el funcionamiento de aplicaciones modernas, ya que permiten almacenar y recuperar grandes volúmenes de información de manera eficiente.

Conceptos básicos I

¿Qué es un sistema de gestión de bases de datos?

- Un sistema de gestión de bases de datos (SGBD) es un software que permite crear, gestionar y manipular bases de datos.
- Proporciona herramientas para definir la estructura de la base de datos, insertar, actualizar y eliminar datos, así como realizar consultas para recuperar información.
- Los SGBD pueden ser clasificados en diferentes tipos, como SGBD relacionales, no relacionales, orientados a objetos, entre otros.
- Los SGBD son fundamentales para el funcionamiento de aplicaciones que requieren almacenamiento y recuperación eficiente de información.
- Proporcionan mecanismos para garantizar la integridad y seguridad de los datos almacenados.
- Algunos ejemplos populares de SGBD incluyen MySQL, PostgreSQL, MongoDB y Oracle Database.

Conceptos básicos I

¿Qué es un modelo de datos?

- Un modelo de datos es una representación abstracta de la estructura y organización de los datos en una base de datos.
- Define cómo se organizan los datos, las relaciones entre ellos y las reglas que rigen su integridad.
- Los modelos de datos pueden ser clasificados en diferentes tipos, como modelos jerárquicos, de red, relacionales, orientados a objetos, entre otros.
- El modelo de datos relacional es uno de los más utilizados y se basa en tablas que contienen filas y columnas.
- Los modelos de datos son fundamentales para el diseño y desarrollo de bases de datos, ya que proporcionan una guía para la creación y manipulación de los datos.
- Un buen modelo de datos ayuda a garantizar la eficiencia y escalabilidad del sistema de gestión de bases de datos.

Conceptos básicos I

¿Qué es un esquema de base de datos?

- Un esquema de base de datos es una descripción formal de la estructura de una base de datos, que incluye las tablas, campos, relaciones y restricciones.
- Define cómo se organizan los datos en la base de datos y cómo se relacionan entre sí.
- El esquema es fundamental para el diseño y desarrollo de la base de datos, ya que proporciona una guía para la creación y manipulación de los datos.
- Un esquema bien diseñado ayuda a garantizar la integridad y eficiencia del sistema de gestión de bases de datos.
- Los esquemas pueden ser representados gráficamente mediante diagramas entidad-relación (ER) o mediante lenguajes de definición de datos (DDL).
- Los cambios en el esquema pueden requerir migraciones o actualizaciones en la base de datos para mantener la coherencia y funcionalidad del sistema.

Conceptos básicos I

¿Qué es un lenguaje de consulta?

- Un lenguaje de consulta es un lenguaje utilizado para interactuar con bases de datos, permitiendo a los usuarios realizar operaciones de búsqueda, inserción, actualización y eliminación de datos.
- Los lenguajes de consulta más comunes son SQL (Structured Query Language) para bases de datos relacionales y MongoDB Query Language (MQL) para bases de datos no relacionales.
- Estos lenguajes proporcionan una sintaxis específica para definir consultas y manipular datos de manera eficiente.
- Los lenguajes de consulta son fundamentales para el desarrollo de aplicaciones que requieren acceso a bases de datos, ya que permiten a los desarrolladores recuperar y manipular información de manera efectiva.
- Un buen conocimiento del lenguaje de consulta es esencial para optimizar el rendimiento y la eficiencia en la interacción con bases de datos.

Conceptos básicos I

¿Qué es una transacción?

- Una transacción es una unidad lógica de trabajo que se ejecuta en una base de datos y que debe cumplir con las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).
- Las transacciones garantizan que las operaciones en la base de datos se realicen de manera segura y coherente, incluso en caso de fallos o errores.
- La atomicidad asegura que todas las operaciones de la transacción se completen con éxito o ninguna se aplique, evitando estados intermedios inconsistentes.
- La consistencia garantiza que la base de datos pase de un estado válido a otro estado válido después de la transacción.
- El aislamiento asegura que las transacciones concurrentes no interfieran entre sí, manteniendo la integridad de los datos.
- La durabilidad garantiza que los cambios realizados por una transacción se persistan en la base de datos, incluso en caso de fallos del sistema.

Conceptos básicos I

¿Qué es la normalización?

- La normalización es un proceso de diseño de bases de datos que busca eliminar redundancias y dependencias innecesarias entre los datos.
- Se basa en una serie de reglas y formas normales que definen cómo deben organizarse los datos en tablas para garantizar la integridad y eficiencia del sistema.
- La normalización ayuda a reducir el espacio de almacenamiento requerido y mejora el rendimiento de las consultas al evitar duplicaciones innecesarias.
- Existen varias formas normales, desde la primera forma normal (1NF) hasta la quinta forma normal (5NF), cada una con sus propias reglas y objetivos.
- Aunque la normalización es importante, en algunos casos puede ser necesario desnormalizar ciertos aspectos para mejorar el rendimiento en aplicaciones específicas.

Conceptos básicos II

¿Qué es la normalización?

- Un buen diseño de base de datos debe equilibrar la normalización con las necesidades de rendimiento y escalabilidad del sistema.

Conceptos básicos I

¿Qué es la denormalización?

- La denormalización es el proceso de introducir redundancias en una base de datos para mejorar el rendimiento de las consultas y reducir la complejidad de las operaciones.
- A diferencia de la normalización, que busca eliminar redundancias, la denormalización puede implicar combinar tablas o agregar columnas adicionales para optimizar el acceso a los datos.
- Aunque la denormalización puede mejorar el rendimiento en ciertas situaciones, también puede aumentar el riesgo de inconsistencias y complicar el mantenimiento de la base de datos.
- Es importante evaluar cuidadosamente cuándo y cómo aplicar la denormalización, considerando las necesidades específicas de la aplicación y el equilibrio entre rendimiento e integridad de los datos.

Conceptos básicos II

¿Qué es la denormalización?

- La denormalización no debe ser vista como una solución general, sino como una herramienta que se utiliza en casos específicos donde se justifica su uso.

Conceptos básicos I

¿Qué es un índice?

- Un índice es una estructura de datos que mejora la velocidad de las operaciones de búsqueda y recuperación de datos en una base de datos.
- Los índices permiten acceder rápidamente a filas específicas en una tabla sin tener que escanear toda la tabla, lo que mejora el rendimiento de las consultas.
- Los índices pueden ser creados en una o más columnas de una tabla y pueden ser únicos o no únicos, dependiendo de si se permite la duplicación de valores.
- Aunque los índices mejoran el rendimiento de las consultas, también pueden aumentar el tiempo de inserción, actualización y eliminación de datos, ya que deben mantenerse actualizados.
- Es importante seleccionar cuidadosamente qué columnas indexar para equilibrar el rendimiento y el costo asociado con la creación y mantenimiento de los índices.

Conceptos básicos I

¿Qué es la integridad referencial?

- La integridad referencial es una propiedad de las bases de datos relacionales que garantiza que las relaciones entre tablas se mantengan consistentes.
- Se asegura de que los valores en una columna de una tabla (clave foránea) correspondan a valores válidos en otra tabla (clave primaria).
- La integridad referencial ayuda a prevenir la creación de registros huérfanos y asegura que las relaciones entre datos sean coherentes.
- Cuando se elimina o actualiza un registro en una tabla, la integridad referencial puede imponer restricciones para evitar que se rompan las relaciones con otras tablas.
- Las restricciones de integridad referencial son definidas mediante claves foráneas y pueden incluir acciones como cascada, restricción o nulificación.

Conceptos básicos I

¿Qué es la seguridad en bases de datos?

- La seguridad en bases de datos se refiere a las medidas y prácticas implementadas para proteger los datos almacenados contra accesos no autorizados, modificaciones o eliminaciones.
- Incluye la autenticación de usuarios, autorización de acceso, cifrado de datos y auditoría de actividades.
- La autenticación verifica la identidad de los usuarios que intentan acceder a la base de datos, mientras que la autorización determina qué acciones pueden realizar.
- El cifrado protege los datos sensibles al convertirlos en un formato ilegible para usuarios no autorizados.
- La auditoría registra las actividades realizadas en la base de datos para detectar posibles violaciones de seguridad o comportamientos sospechosos.
- La seguridad en bases de datos es esencial para proteger la confidencialidad, integridad y disponibilidad de los datos almacenados.

Conceptos básicos I

¿Qué es la escalabilidad en bases de datos?

- La escalabilidad en bases de datos se refiere a la capacidad de un sistema para manejar un aumento en la carga de trabajo sin comprometer el rendimiento.
- Puede ser vertical (aumentando los recursos de hardware en un solo servidor) u horizontal (agregando más servidores al sistema).
- La escalabilidad es crucial para aplicaciones que experimentan un crecimiento en el volumen de datos o en el número de usuarios.
- Un diseño escalable permite que la base de datos se adapte a las necesidades cambiantes del negocio y soporte un mayor volumen de transacciones.
- Las técnicas de particionamiento, replicación y uso de cachés son comunes para lograr escalabilidad en bases de datos.

Conceptos básicos I

¿Qué es la disponibilidad en bases de datos?

- La disponibilidad en bases de datos se refiere a la capacidad del sistema para estar operativo y accesible para los usuarios en todo momento.
- Es esencial para aplicaciones críticas donde el tiempo de inactividad puede resultar en pérdidas significativas.
- La alta disponibilidad se logra mediante técnicas como replicación, clustering y uso de copias de seguridad.
- Los sistemas de bases de datos deben ser diseñados para tolerar fallos y garantizar que los datos estén disponibles incluso en caso de problemas técnicos.
- La monitorización constante y la implementación de planes de recuperación ante desastres son prácticas comunes para mantener la disponibilidad.

Conceptos básicos I

¿Qué es la recuperación ante desastres en bases de datos?

- La recuperación ante desastres en bases de datos se refiere a las estrategias y procedimientos implementados para restaurar el sistema y los datos después de un fallo catastrófico.
- Incluye la creación de copias de seguridad regulares, la replicación de datos y la implementación de planes de contingencia.
- Un buen plan de recuperación ante desastres debe definir los pasos a seguir en caso de pérdida de datos, fallos del sistema o desastres naturales.
- La recuperación debe ser rápida y eficiente para minimizar el tiempo de inactividad y la pérdida de datos.
- Las pruebas periódicas del plan de recuperación son esenciales para garantizar su efectividad y adaptabilidad a diferentes escenarios.

Conceptos básicos I

¿Qué es la replicación en bases de datos?

- La replicación en bases de datos es el proceso de copiar y mantener actualizados los datos en múltiples servidores o ubicaciones.
- Permite mejorar la disponibilidad, escalabilidad y rendimiento del sistema al distribuir la carga de trabajo entre varios nodos.
- Existen diferentes tipos de replicación, como la replicación maestra-esclavo, maestra-maestra y basada en eventos.
- La replicación también puede ser síncrona (los cambios se aplican en todos los nodos al mismo tiempo) o asíncrona (los cambios se propagan con un retraso).
- Es una técnica comúnmente utilizada para garantizar la alta disponibilidad y tolerancia a fallos en sistemas distribuidos.

Conceptos básicos I

¿Qué es la partición de datos?

- La partición de datos es el proceso de dividir una base de datos en partes más pequeñas y manejables, llamadas particiones.
- Permite distribuir los datos en múltiples servidores o nodos para mejorar el rendimiento y la escalabilidad del sistema.
- Existen diferentes estrategias de partición, como la partición horizontal (dividir filas) y la partición vertical (dividir columnas).
- La partición ayuda a reducir el tiempo de respuesta de las consultas al limitar la cantidad de datos que se deben procesar en cada operación.
- Es una técnica comúnmente utilizada en sistemas distribuidos para manejar grandes volúmenes de datos y mejorar la eficiencia del acceso a la información.

Conceptos básicos I

¿Qué es la consistencia eventual?

- La consistencia eventual es un modelo de consistencia utilizado en sistemas distribuidos donde los datos pueden no estar inmediatamente consistentes en todos los nodos.
- Permite que las actualizaciones se propaguen de manera asíncrona, lo que mejora la disponibilidad y el rendimiento del sistema.
- En este modelo, se garantiza que, eventualmente, todos los nodos convergerán hacia un estado consistente, pero no se garantiza la consistencia inmediata.
- Es comúnmente utilizado en bases de datos no relacionales y sistemas distribuidos donde la latencia y la disponibilidad son prioritarias sobre la consistencia inmediata.
- La consistencia eventual es adecuada para aplicaciones donde la tolerancia a fallos y la escalabilidad son más importantes que la consistencia inmediata de los datos.

Conceptos básicos I

¿Qué es la replicación de datos?

- La replicación de datos es el proceso de copiar y mantener actualizados los datos en múltiples servidores o ubicaciones.
- Permite mejorar la disponibilidad, escalabilidad y rendimiento del sistema al distribuir la carga de trabajo entre varios nodos.
- Existen diferentes tipos de replicación, como la replicación maestra-esclavo, maestra-maestra y basada en eventos.
- La replicación también puede ser síncrona (los cambios se aplican en todos los nodos al mismo tiempo) o asíncrona (los cambios se propagan con un retraso).
- Es una técnica comúnmente utilizada para garantizar la alta disponibilidad y tolerancia a fallos en sistemas distribuidos.

Conceptos básicos I

¿Qué es la tolerancia a fallos?

- La tolerancia a fallos es la capacidad de un sistema para continuar operando correctamente incluso en presencia de fallos o errores.
- Implica diseñar el sistema de manera que pueda detectar y recuperarse de fallos sin interrumpir el servicio.
- La tolerancia a fallos se logra mediante técnicas como replicación, redundancia, monitoreo y recuperación ante desastres.
- Es esencial para aplicaciones críticas donde la disponibilidad y continuidad del servicio son prioritarias.
- Un sistema tolerante a fallos puede manejar situaciones como fallos de hardware, errores de software o problemas de red sin afectar la experiencia del usuario.

Conceptos básicos I

¿Qué es la escalabilidad horizontal?

- La escalabilidad horizontal es la capacidad de un sistema para aumentar su capacidad agregando más nodos o servidores al sistema.
- A diferencia de la escalabilidad vertical, que implica aumentar los recursos de hardware en un solo servidor, la escalabilidad horizontal distribuye la carga de trabajo entre múltiples nodos.
- Permite manejar un mayor volumen de datos y usuarios sin comprometer el rendimiento del sistema.
- Es una técnica comúnmente utilizada en sistemas distribuidos y aplicaciones web para mejorar la disponibilidad y tolerancia a fallos.
- La escalabilidad horizontal es esencial para aplicaciones que experimentan un crecimiento rápido en el volumen de datos o en el número de usuarios.

Conceptos básicos I

¿Qué es la escalabilidad vertical?

- La escalabilidad vertical es la capacidad de un sistema para aumentar su capacidad mejorando los recursos de hardware en un solo servidor.
- Implica agregar más memoria, CPU o almacenamiento al servidor existente para manejar una mayor carga de trabajo.
- Aunque la escalabilidad vertical puede ser efectiva para aumentar el rendimiento, tiene limitaciones físicas y económicas.
- Es comúnmente utilizada en sistemas donde la carga de trabajo no es extremadamente alta y se puede manejar con un solo servidor mejorado.
- La escalabilidad vertical es adecuada para aplicaciones que requieren un alto rendimiento en un entorno controlado y no necesitan distribuir la carga entre múltiples nodos.

Conceptos básicos I

¿Qué es la concurrencia en bases de datos?

- La concurrencia en bases de datos se refiere a la capacidad de múltiples usuarios o procesos para acceder y manipular los datos simultáneamente.
- Es esencial para aplicaciones donde varios usuarios pueden realizar operaciones al mismo tiempo sin interferir entre sí.
- La gestión de la concurrencia implica el uso de mecanismos como bloqueos, transacciones y control de versiones para garantizar la integridad de los datos.
- La concurrencia puede mejorar el rendimiento del sistema al permitir que múltiples operaciones se realicen en paralelo.
- Sin embargo, también puede introducir desafíos como condiciones de carrera, bloqueos y problemas de consistencia si no se maneja adecuadamente.

Tema 2

Diseño de modelos de datos

Diseño de modelos de datos I

¿Qué es el diseño de modelos de datos?

- El diseño de modelos de datos es el proceso de crear una representación abstracta de la estructura y organización de los datos en una base de datos.
- Implica identificar las entidades, atributos y relaciones entre los datos para garantizar una representación coherente y eficiente.
- Un buen diseño de modelo de datos ayuda a optimizar el rendimiento, la escalabilidad y la mantenibilidad del sistema de gestión de bases de datos.
- Los modelos de datos pueden ser representados gráficamente mediante diagramas entidad-relación (ER) o mediante lenguajes de definición de datos (DDL).
- El diseño de modelos de datos es fundamental para el desarrollo exitoso de aplicaciones que requieren almacenamiento y recuperación eficiente de información.

Diseño de modelos de datos I

¿Qué es un diagrama entidad-relación (ER)?

- Un diagrama entidad-relación (ER) es una representación gráfica de un modelo de datos que muestra las entidades, atributos y relaciones entre ellas.
- Utiliza símbolos específicos para representar entidades (rectángulos), atributos (óvalos) y relaciones (rombos).
- Los diagramas ER son herramientas útiles para visualizar y comunicar el diseño de la base de datos a los desarrolladores y partes interesadas.
- Ayudan a identificar las dependencias entre los datos y a definir las claves primarias y foráneas necesarias para mantener la integridad referencial.
- Los diagramas ER son una parte fundamental del proceso de diseño de bases de datos y se utilizan ampliamente en la ingeniería de software.

Diseño de modelos de datos I

¿Qué es la normalización de bases de datos?

- La normalización de bases de datos es el proceso de organizar los datos en una base de datos para reducir redundancias y dependencias innecesarias.
- Se basa en una serie de reglas y formas normales que definen cómo deben estructurarse las tablas y las relaciones entre ellas.
- La normalización ayuda a mejorar la integridad de los datos, optimizar el rendimiento y facilitar el mantenimiento del sistema.
- Existen varias formas normales, desde la primera forma normal (1NF) hasta la quinta forma normal (5NF), cada una con sus propias reglas y objetivos.
- Aunque la normalización es importante, en algunos casos puede ser necesario desnormalizar ciertos aspectos para mejorar el rendimiento en aplicaciones específicas.

Diseño de modelos de datos I

¿Qué es la denormalización de bases de datos?

- La denormalización de bases de datos es el proceso de introducir redundancias en una base de datos para mejorar el rendimiento de las consultas y reducir la complejidad de las operaciones.
- A diferencia de la normalización, que busca eliminar redundancias, la denormalización puede implicar combinar tablas o agregar columnas adicionales para optimizar el acceso a los datos.
- Aunque la denormalización puede mejorar el rendimiento en ciertas situaciones, también puede aumentar el riesgo de inconsistencias y complicar el mantenimiento de la base de datos.
- Es importante evaluar cuidadosamente cuándo y cómo aplicar la denormalización, considerando las necesidades específicas de la aplicación y el equilibrio entre rendimiento e integridad de los datos.

Diseño de modelos de datos II

¿Qué es la denormalización de bases de datos?

- La denormalización no debe ser vista como una solución general, sino como una herramienta que se utiliza en casos específicos donde se justifica su uso.

Diseño de modelos de datos I

¿Qué es un modelo de datos relacional?

- Un modelo de datos relacional es un tipo de modelo de datos que organiza los datos en tablas (relaciones) que se pueden relacionar entre sí mediante claves primarias y foráneas.
- Cada tabla contiene filas (registros) y columnas (atributos), donde cada fila representa una entidad y cada columna representa un atributo de esa entidad.
- El modelo relacional es ampliamente utilizado en sistemas de gestión de bases de datos (SGBD) como MySQL, PostgreSQL y Oracle Database.
- Proporciona un enfoque estructurado y flexible para el almacenamiento y recuperación de datos, permitiendo realizar consultas complejas mediante el uso del lenguaje SQL.
- La integridad referencial y las restricciones son fundamentales en el modelo relacional para garantizar la coherencia y validez de los datos almacenados.

Diseño de modelos de datos I

¿Qué es un modelo de datos no relacional?

- Un modelo de datos no relacional es un tipo de modelo de datos que no sigue el enfoque tabular del modelo relacional y permite una mayor flexibilidad en la estructura de los datos.
- Los modelos no relacionales incluyen bases de datos orientadas a documentos, clave-valor, columnares y de grafos, entre otros.
- Estos modelos son ideales para aplicaciones que requieren escalabilidad horizontal, alta disponibilidad y manejo de grandes volúmenes de datos no estructurados o semiestructurados.
- Ejemplos populares de bases de datos no relacionales incluyen MongoDB (orientada a documentos), Redis (clave-valor) y Neo4j (de grafos).
- Aunque los modelos no relacionales ofrecen flexibilidad y rendimiento, pueden carecer de algunas características avanzadas de integridad y consistencia presentes en los modelos relacionales.

Diseño de modelos de datos I

¿Qué es un modelo de datos orientado a objetos?

- Un modelo de datos orientado a objetos es un enfoque que representa los datos como objetos, similar a la programación orientada a objetos.
- Los objetos encapsulan tanto los datos (atributos) como el comportamiento (métodos) relacionados con esos datos.
- Este modelo permite una mayor flexibilidad y reutilización de código, ya que los objetos pueden heredar propiedades y comportamientos de otros objetos.
- Es utilizado en bases de datos orientadas a objetos como ObjectDB y db4o, que permiten almacenar y recuperar objetos directamente sin necesidad de convertirlos a un formato tabular.
- Aunque el modelo orientado a objetos ofrece ventajas en términos de diseño y abstracción, puede ser menos eficiente en términos de rendimiento en comparación con los modelos relacionales para ciertas operaciones.

Tema 3

Sistemas de gestión de bases de datos

Sistemas de gestión de bases de datos I

¿Qué es un sistema de gestión de bases de datos (SGBD)?

- Un sistema de gestión de bases de datos (SGBD) es un software que permite crear, gestionar y manipular bases de datos.
- Proporciona herramientas para definir la estructura de la base de datos, insertar, actualizar y eliminar datos, así como realizar consultas para recuperar información.
- Los SGBD pueden ser clasificados en diferentes tipos, como SGBD relacionales, no relacionales, orientados a objetos, entre otros.
- Los SGBD son fundamentales para el funcionamiento de aplicaciones que requieren almacenamiento y recuperación eficiente de información.
- Proporcionan mecanismos para garantizar la integridad y seguridad de los datos almacenados.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD relacional?

- Un SGBD relacional es un tipo de sistema de gestión de bases de datos que organiza los datos en tablas (relaciones) que se pueden relacionar entre sí mediante claves primarias y foráneas.
- Utiliza el lenguaje SQL (Structured Query Language) para definir, manipular y consultar los datos almacenados en las tablas.
- Los SGBD relacionales son ampliamente utilizados en aplicaciones empresariales y sistemas críticos debido a su capacidad para garantizar la integridad y consistencia de los datos.
- Ejemplos populares de SGBD relacionales incluyen MySQL, PostgreSQL, Oracle Database y Microsoft SQL Server.
- Proporcionan características avanzadas como transacciones, control de concurrencia y seguridad de datos.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD no relacional?

- Un SGBD no relacional es un tipo de sistema de gestión de bases de datos que no sigue el enfoque tabular del modelo relacional y permite una mayor flexibilidad en la estructura de los datos.
- Los SGBD no relacionales incluyen bases de datos orientadas a documentos, clave-valor, columnares y de grafos, entre otros.
- Estos SGBD son ideales para aplicaciones que requieren escalabilidad horizontal, alta disponibilidad y manejo de grandes volúmenes de datos no estructurados o semiestructurados.
- Ejemplos populares de SGBD no relacionales incluyen MongoDB (orientada a documentos), Redis (clave-valor) y Neo4j (de grafos).
- Aunque los SGBD no relacionales ofrecen flexibilidad y rendimiento, pueden carecer de algunas características avanzadas de integridad y consistencia presentes en los SGBD relacionales.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD orientado a objetos?

- Un SGBD orientado a objetos es un tipo de sistema de gestión de bases de datos que representa los datos como objetos, similar a la programación orientada a objetos.
- Los objetos encapsulan tanto los datos (atributos) como el comportamiento (métodos) relacionados con esos datos.
- Este enfoque permite una mayor flexibilidad y reutilización de código, ya que los objetos pueden heredar propiedades y comportamientos de otros objetos.
- Los SGBD orientados a objetos son utilizados en aplicaciones que requieren una representación compleja de los datos y relaciones, como sistemas CAD/CAM y aplicaciones multimedia.
- Ejemplos de SGBD orientados a objetos incluyen ObjectDB y db4o.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD distribuido?

- Un SGBD distribuido es un sistema de gestión de bases de datos que se ejecuta en múltiples servidores o nodos, distribuyendo los datos y las operaciones entre ellos.
- Permite mejorar la escalabilidad, disponibilidad y tolerancia a fallos al distribuir la carga de trabajo entre varios nodos.
- Los SGBD distribuidos pueden ser relacionales o no relacionales y utilizan técnicas como replicación y particionamiento para gestionar los datos.
- Ejemplos de SGBD distribuidos incluyen Apache Cassandra, Google Spanner y Amazon DynamoDB.
- Estos sistemas son ideales para aplicaciones que requieren alta disponibilidad y rendimiento en entornos distribuidos.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD en la nube?

- Un SGBD en la nube es un sistema de gestión de bases de datos que se ofrece como un servicio a través de plataformas en la nube, como Amazon Web Services (AWS), Microsoft Azure o Google Cloud Platform.
- Permite a las organizaciones escalar sus bases de datos sin necesidad de gestionar la infraestructura subyacente, lo que reduce costos y complejidad operativa.
- Los SGBD en la nube pueden ser relacionales o no relacionales y ofrecen características como alta disponibilidad, copias de seguridad automáticas y recuperación ante desastres.
- Ejemplos populares de SGBD en la nube incluyen Amazon RDS (relacional), Google Cloud Firestore (no relacional) y Azure Cosmos DB (multi-modelo).
- Estos sistemas son ideales para aplicaciones que requieren flexibilidad, escalabilidad y acceso global a los datos.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD de código abierto?

- Un SGBD de código abierto es un sistema de gestión de bases de datos cuyo código fuente está disponible públicamente y puede ser modificado y distribuido por la comunidad.
- Estos SGBD suelen ser gratuitos y ofrecen una amplia gama de características y funcionalidades, lo que los hace populares entre desarrolladores y empresas.
- Ejemplos populares de SGBD de código abierto incluyen MySQL, PostgreSQL, MongoDB y MariaDB.
- Los SGBD de código abierto fomentan la colaboración y la innovación, ya que la comunidad puede contribuir al desarrollo y mejora del software.
- Aunque son gratuitos, las empresas pueden optar por versiones comerciales con soporte técnico adicional y características avanzadas.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD comercial?

- Un SGBD comercial es un sistema de gestión de bases de datos desarrollado y vendido por una empresa, que ofrece soporte técnico, actualizaciones y características avanzadas.
- Estos SGBD suelen requerir licencias de uso y pueden ser más costosos que las alternativas de código abierto.
- Ejemplos populares de SGBD comerciales incluyen Oracle Database, Microsoft SQL Server y IBM Db2.
- Los SGBD comerciales suelen ofrecer características avanzadas como optimización de consultas, herramientas de administración y seguridad mejorada.
- Son ideales para empresas que requieren soporte técnico confiable y características específicas para aplicaciones críticas.

Sistemas de gestión de bases de datos I

¿Qué es un SGBD embebido?

- Un SGBD embebido es un sistema de gestión de bases de datos diseñado para ser integrado en aplicaciones o dispositivos, proporcionando almacenamiento y recuperación de datos sin necesidad de un servidor independiente.
- Estos SGBD son ligeros y eficientes, lo que los hace ideales para aplicaciones móviles, dispositivos IoT y sistemas embebidos.
- Ejemplos populares de SGBD embebidos incluyen SQLite, Berkeley DB y H2 Database.
- Los SGBD embebidos permiten a los desarrolladores incorporar capacidades de base de datos directamente en sus aplicaciones, mejorando la portabilidad y el rendimiento.
- Aunque son menos potentes que los SGBD tradicionales, ofrecen una solución efectiva para aplicaciones que requieren almacenamiento local de datos.

Tema 4

MySQL y MariaDB

MySQL y MariaDB I

¿Qué es MySQL?

- MySQL es un sistema de gestión de bases de datos relacional de código abierto, ampliamente utilizado en aplicaciones web y empresariales.
- Es conocido por su rendimiento, escalabilidad y facilidad de uso, lo que lo convierte en una opción popular para desarrolladores y administradores de bases de datos.
- MySQL utiliza el lenguaje SQL para definir, manipular y consultar los datos almacenados en las tablas.
- Ofrece características avanzadas como transacciones, control de concurrencia, replicación y seguridad de datos.
- MySQL es compatible con una amplia variedad de sistemas operativos y lenguajes de programación, lo que facilita su integración en diferentes entornos.

MySQL y MariaDB I

¿Qué es MariaDB?

- MariaDB es un sistema de gestión de bases de datos relacional de código abierto, creado como un fork de MySQL por los desarrolladores originales de MySQL.
- Se desarrolló para garantizar la continuidad del proyecto MySQL después de su adquisición por parte de Oracle Corporation.
- MariaDB es compatible con MySQL, lo que significa que las aplicaciones diseñadas para MySQL pueden funcionar sin modificaciones en MariaDB.
- Ofrece características adicionales y mejoras en el rendimiento, escalabilidad y seguridad en comparación con MySQL.
- MariaDB es utilizado por muchas organizaciones y proyectos de código abierto como una alternativa a MySQL, manteniendo la filosofía de código abierto y comunidad.

MySQL y MariaDB I

¿Cuáles son las diferencias entre MySQL y MariaDB?

- Aunque MySQL y MariaDB son similares, existen algunas diferencias clave entre ellos:
- **Licencia:** MySQL es propiedad de Oracle Corporation y tiene una licencia dual (comercial y GPL), mientras que MariaDB es completamente de código abierto bajo la licencia GPL.
- **Características:** MariaDB incluye características adicionales como nuevos motores de almacenamiento, mejoras en el rendimiento y nuevas funciones de seguridad que no están presentes en MySQL.
- **Compatibilidad:** MariaDB es compatible con MySQL, lo que significa que las aplicaciones diseñadas para MySQL pueden funcionar sin modificaciones en MariaDB, pero no al revés.

MySQL y MariaDB II

¿Cuáles son las diferencias entre MySQL y MariaDB?

- **Desarrollo:** MariaDB es desarrollado por la comunidad y tiene un enfoque más centrado en la comunidad, mientras que MySQL es desarrollado por Oracle Corporation.
- **Rendimiento:** En algunos casos, MariaDB puede ofrecer un mejor rendimiento y escalabilidad en comparación con MySQL debido a sus optimizaciones y mejoras.

Tema 5

Instalación y configuración de MySQL o MariaDB

Instalación y configuración de MySQL o MariaDB I

¿Cómo instalar MySQL o MariaDB?

- La instalación de MySQL o MariaDB puede variar según el sistema operativo, pero generalmente sigue estos pasos:
- **Descarga:** Visite el sitio web oficial de MySQL (<https://www.mysql.com/>) o MariaDB (<https://mariadb.org/>) para descargar la versión adecuada para su sistema operativo.
- **Instalación:** Siga las instrucciones de instalación proporcionadas por el instalador, que generalmente incluyen la selección de opciones de configuración, como la ubicación de los archivos de datos y la configuración de la contraseña de root.
- **Configuración:** Después de la instalación, es importante configurar el SGBD para garantizar la seguridad y el rendimiento. Esto puede incluir la creación de usuarios y roles, la configuración de permisos y la optimización de parámetros de rendimiento.

Instalación y configuración de MySQL o MariaDB II

¿Cómo instalar MySQL o MariaDB?

- **Prueba:** Una vez instalado y configurado, es recomendable realizar pruebas para asegurarse de que el SGBD esté funcionando correctamente y que pueda aceptar conexiones y ejecutar consultas.

Instalación y configuración de MySQL o MariaDB I

¿Cómo configurar MySQL o MariaDB?

La configuración de MySQL o MariaDB implica ajustar varios parámetros para optimizar el rendimiento, la seguridad y la funcionalidad del sistema. Algunos aspectos clave de la configuración incluyen:

- **Seguridad:** Configure la contraseña de root, cree usuarios con permisos específicos y deshabilite el acceso remoto si no es necesario para mejorar la seguridad.
- **Rendimiento:** Ajuste parámetros como el tamaño de la caché, el número de conexiones simultáneas y la configuración de los motores de almacenamiento para optimizar el rendimiento del SGBD.
- **Replicación:** Si está utilizando replicación, configure los parámetros relacionados con la replicación, como el servidor maestro y esclavo, y asegúrese de que la replicación esté funcionando correctamente.

Instalación y configuración de MySQL o MariaDB II

¿Cómo configurar MySQL o MariaDB?

- **Copia de seguridad:** Configure las opciones de copia de seguridad para garantizar que los datos estén protegidos y puedan ser restaurados en caso de pérdida o fallo del sistema.
- **Monitoreo:** Configure herramientas de monitoreo para supervisar el rendimiento y la salud del SGBD, lo que le permitirá detectar y resolver problemas de manera proactiva.

Prueba de clientes y conexión a MySQL

- Para probar la conexión a MySQL o MariaDB, puede utilizar clientes de línea de comandos como 'mysql' o herramientas gráficas como MySQL Workbench o phpMyAdmin.
- Asegúrese de que el servidor MySQL o MariaDB esté en ejecución y que las credenciales de usuario sean correctas.
- Intente conectarse al servidor utilizando el cliente de su elección y ejecute consultas básicas para verificar que la conexión sea exitosa y que pueda interactuar con la base de datos.

- Para conectarse a MySQL o MariaDB desde aplicaciones Java, puede utilizar el conector JDBC proporcionado por MySQL o MariaDB.
- Para aplicaciones Python, puede utilizar bibliotecas como 'mysql-connector-python' o 'PyMySQL' para establecer la conexión y ejecutar consultas.
- Asegúrese de instalar las bibliotecas necesarias y configurar las credenciales de conexión correctamente en su aplicación para garantizar una conexión exitosa al SGBD.
- Una vez establecida la conexión, puede ejecutar consultas SQL desde su aplicación para interactuar con la base de datos y manipular los datos según sea necesario.

- PlantUML es una herramienta de código abierto que permite crear diagramas a partir de texto, incluyendo diagramas entidad-relación (ER) para modelar bases de datos.
- Utiliza una sintaxis simple y fácil de aprender para definir entidades, atributos y relaciones entre ellos.
- PlantUML puede generar diagramas en varios formatos, como PNG, SVG y PDF, lo que facilita su integración en documentación y presentaciones.
- Es una herramienta útil para diseñar y comunicar la estructura de una base de datos de manera visual y clara.

Ejemplo de modelo ER con Plantuml

- A continuación se muestra un ejemplo de cómo modelar una base de datos utilizando PlantUML para crear un diagrama entidad-relación (ER):

```
@startuml
entity "Cliente" {
    *id : int
    nombre : string
    email : string
}
entity "Pedido" {
    *id : int
    fecha : date
    total : decimal
}
Cliente ||--o{ Pedido : realiza
@enduml
```

- En este ejemplo, se definen dos entidades: "Cliente" y "Pedido". La entidad "Cliente" tiene atributos como "id", "nombre" y "email", mientras que la entidad "Pedido" tiene atributos como "id", "fecha" y "total". La relación entre ellas es "realiza".